

1. What is AWS Lambda?

AWS Lambda is a serverless compute service that lets you run code without managing servers.

With traditional servers, you need to provision servers, install software, manage CPU/memory, apply patches, monitor infrastructure, and handle scaling. With Lambda, AWS manages the underlying infrastructure and you focus on your code.

Lambda = Run your code when something happens, without managing a server.

Event → Lambda → Your Code → Result

2. Lambda vs. Traditional Server

Traditional Approach

```
EC2 Server
├── Operating System
├── Runtime / Dependencies
├── Application
└── Your Code
```

You are responsible for managing the server and application environment.

Lambda Approach

```
AWS Lambda
├── Your Function
│   └── Your Code
```

AWS manages the infrastructure, operating system, runtime environment, and scaling.

3. What is a Lambda Function?

A **Lambda Function** is the unit of code that Lambda executes.

Example in Python:

```
def lambda_handler(event, context):
    print("Hello from Lambda!")

    return {
        "statusCode": 200,
        "body": "Hello World"
    }
```

Lambda provides two important inputs:

Input

Purpose

event Information about what triggered the function
context Information about the current Lambda execution

Think of it as:

Function = What should happen?

Lambda Service = Where and how the code runs?

4. Lambda is Event Driven

Lambda is commonly used in **event-driven architectures**.
Something happens → Lambda is triggered → Code executes.
Common event sources include:

- Amazon S3
- Amazon API Gateway
- Amazon SQS
- Amazon SNS
- Amazon EventBridge
- DynamoDB Streams
- Kinesis

Example:

User uploads file

↓

S3

↓

Object Created Event

↓

Lambda

Event will look like this

```
{
  "Records": [
    {
      "eventVersion": "2.1",
      "eventSource": "aws:s3",
      "awsRegion": "ap-south-1",
      "eventName": "ObjectCreated:Put",
      "s3": {
        "s3SchemaVersion": "1.0",
        "bucket": {
          "name": "dvs-technology-input",
          "arn": "arn:aws:s3:::dvs-technology-input"
        },
        "object": {
          "key": "batch-purchases.json",
          "size": 4096,
          "eTag": "b1946ac92492d2347c6235b4d261184"
        }
      }
    }
  ]
}
```

↓

5. What Triggers a Lambda Function?

A Lambda function does not normally run continuously. Something **invokes** it.

S3

S3 → Object Created Event → Lambda

SQS

SQS → Message → Lambda → Process Message

6. Lambda Invocation Models

There are three important invocation patterns.

6.1 Synchronous Invocation

The caller waits for Lambda to finish and receives the response.

```
Client
  ↓ Request
Lambda
  ↓ Response
Client
```

Typical use case: **APIs and request/response applications.**

6.2 Asynchronous Invocation

The event is delivered to Lambda without the original caller waiting for the function's result.

```
Event Source
  ↓
Lambda
  ↓
Background Processing
```

Common examples:

- S3
- SNS

6.3 Event Source Mapping

For queue/stream-based services, Lambda can use an **event source mapping** to retrieve records and invoke the function.

Common examples:

- SQS

- Kinesis
- DynamoDB Streams

SQS Queue
↓
Event Source Mapping
↓
Lambda
↓
Process Message

7. Lambda Scaling

One of Lambda's biggest advantages is **automatic scaling**.

If one request arrives:

Request → Lambda Execution

If 100 requests arrive concurrently, Lambda can create multiple execution environments:

Request 1 → Lambda #1
Request 2 → Lambda #2
Request 3 → Lambda #3
...
Request 100 → Lambda #100

You don't manually provision 100 servers.

More events → More concurrent Lambda executions

Fewer events → Fewer executions

Actual concurrency is subject to AWS Lambda concurrency limits and configuration.

8. Lambda Execution Environment

When Lambda invokes your function, AWS provides an **execution environment** containing your code and runtime.

Lambda Execution Environment

Your Function Code
Runtime Python / Java / Node.js

Lambda may reuse an existing execution environment for subsequent invocations.

However:

Never assume that an execution environment will always be reused.

9. Cold Start

If Lambda needs a new execution environment, it must initialize the environment before running your function. This can introduce additional startup latency.

```
First Invocation
  ↓
Create Environment
  ↓
Initialize Runtime
  ↓
Run Function
```

A later invocation may use an existing environment:

```
Invocation
  ↓
Existing Environment
  ↓
Run Function
```

This can have lower startup latency.

10. Lambda is Stateless

Lambda functions should be designed to be **stateless**.

Do not depend on local memory or local files being available between invocations.

Store persistent data in external services:

```
Lambda
├──→ S3
├──→ DynamoDB
├──→ RDS
└──→ ElastiCache
```

Lambda performs the work; external AWS services store the persistent data.

11. Lambda Timeout

Every Lambda invocation has a maximum execution duration.

If the function exceeds the configured timeout, Lambda terminates the invocation.

```
Lambda Starts
  ↓
Execute Code
  └──→ Success
```

└─ Timeout

Choose the timeout according to the workload.

Examples:

- API function → Usually short execution
- Image processing → Potentially longer execution
- Very long-running processing → Consider ECS, AWS Batch, Step Functions, or another suitable service

Lambda is designed primarily for **short-lived, event-driven workloads**.

12. Lambda Memory and CPU

When configuring Lambda, you specify the function's **memory allocation**.

Memory allocation also influences the amount of CPU available.

```
More Memory
  ↓
More CPU
  ↓
Potentially Faster Execution
```

More memory generally increases the price per unit of execution time, so choose the configuration based on **performance testing and cost optimization**, rather than simply choosing the largest value.

13. Lambda and IAM

Lambda often needs permission to access other AWS services.

For example:

```
Lambda
├─ Read from S3
├─ Write to DynamoDB
├─ Send message to SQS
└─ Publish to SNS
```

These permissions are typically provided through a **Lambda execution role**.

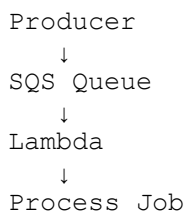
```
Lambda Function
  ↓
IAM Execution Role
├─ s3:GetObject
├─ dynamodb:PutItem
└─ sqs:SendMessage
```

Follow the **principle of least privilege**:

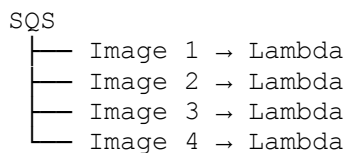
Give the Lambda function only the permissions it actually needs.

14. Lambda + SQS

Lambda and SQS are commonly combined for reliable asynchronous processing.



For example, an image-processing system can place jobs into SQS:



SQS provides a buffer between producers and consumers and helps decouple application components.

15. Lambda + S3

A common beginner use case is processing files uploaded to S3.



Lambda can be used to:

- Resize images
 - Extract metadata
 - Validate files
 - Convert documents
 - Start RAG preprocessing
 - Generate thumbnails
-

16. Common Lambda Use Cases

Use Case	Example
API Backend	Process API Gateway requests
File Processing	Process S3 uploads
Queue Processing	Process SQS messages
Notifications	Publish/send notifications
Automation	Perform maintenance or automation tasks
Scheduled Jobs	Run code periodically
Data Transformation	Transform incoming records
RAG Pipelines	Preprocess documents and generate embeddings
Event Processing	React to EventBridge events

17. What Lambda is NOT Good For

Lambda is not the best solution for every workload.

Be cautious when you need:

- Long-running processes
- Persistent local state
- Specialized server configurations
- Applications requiring continuously running processes
- Workloads that exceed Lambda's execution limits
- Highly predictable latency where cold-start considerations are critical

Depending on the workload, consider:

- **ECS**
- **EKS**
- **EC2**
- **AWS Batch**
- **AWS Step Functions**
- **AWS Glue**

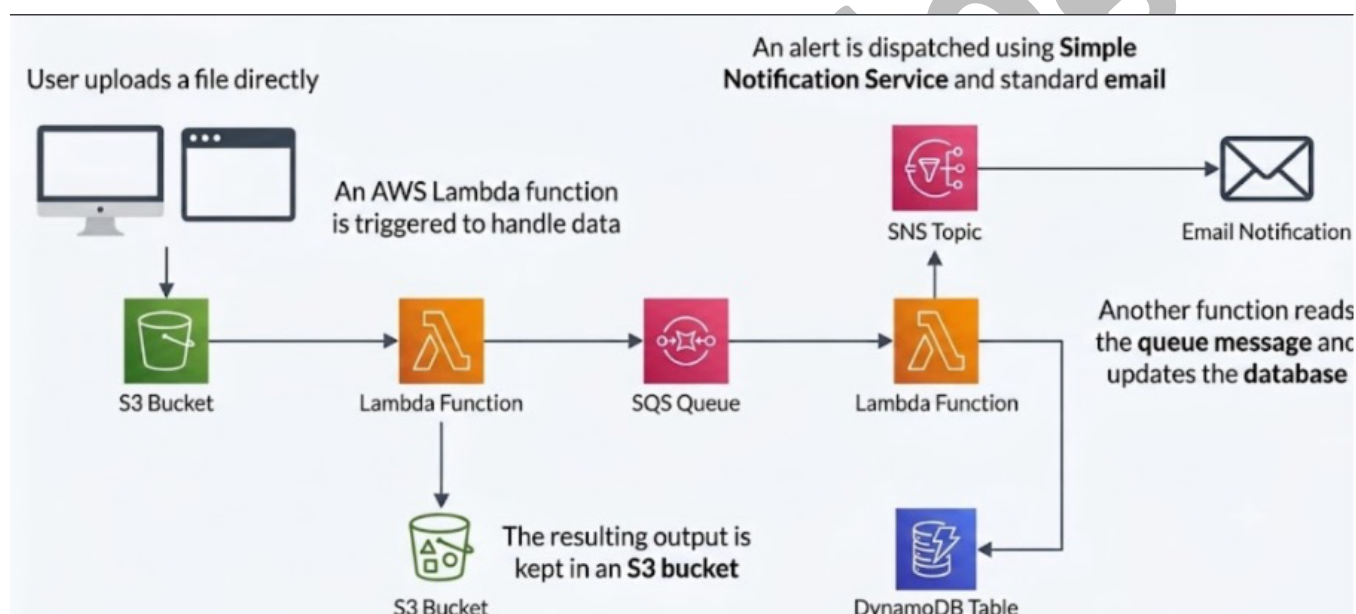
18. Lambda vs. ECS

Lambda and ECS solve different problems.

Feature	AWS Lambda	Amazon ECS
Infrastructure Management	Fully managed serverless compute	Managed container orchestration
Execution Model	Invocation/event based	Container/task based
Long-Running Application	✗ Not ideal	✓ Excellent
Containerized Apps	✓ Supports container images	✓ Designed for containers

Feature	AWS Lambda	Amazon ECS
Automatic Scaling	✔ Built in	✔ Supports Service Auto Scaling
Server Management	No server management	Depends on Fargate or EC2
Best For	Event-driven workloads	APIs, services, workers, containers
Execution Duration	Time-limited	Suitable for long-running services
Typical Example	Process an S3 upload	Run a REST API continuously

Event Driven Architecture:



Why sqs Queue and not processing in first lambda itself:

- The SQS Queue holds the purchase messages safely in line. If 1,000 files are uploaded at the exact same time, the queue acts as a shock absorber so your downstream database isn't overwhelmed.

1. LAB: Deployment Checklist:

- Create three S3 buckets (input, validated, rejected). (Enable public access- incase we want to access it from our machines)
 - dvs-<STUDENT_NAME>-retail-data-input
 - dvs-<STUDENT_NAME>-retail-data-validated
 - dvs-<STUDENT_NAME>-retail-data-rejected
- Create Dynamodb table
 - retail-purchases(customerid (String), purchasedate (String))

- Create an SQS Queue and an SNS Topic and subscription (with email subscription confirmed).
 - dvs-technology-<STUDENT_NAME>-sqs-queue
 - dvs-technology-<STUDENT_NAME>-sns-topic

Here is how you can create an SQS queue quickly using the AWS Management Console:

- Navigate to SQS: Log in to the AWS Management Console, type SQS into the search bar, and select Simple Queue Service.
- Start Creation: Click the orange Create queue button.
- Select Type and Name: Choose your queue type—Standard (best for high throughput) or FIFO (if strict ordering is required)—and enter a name for your queue.
- Create: You can safely leave all the other configuration settings as their defaults for a basic setup. Scroll to the very bottom of the page and click Create queue.

Here is how to create the SNS topic and set it up to send emails to your inbox for this pipeline:

- Navigate to SNS: In the AWS Console, search for SNS and open the Simple Notification Service.
- Create the Topic: On the left sidebar, click Topics, then click the orange Create topic button. Select Standard as the type, give it a name (like dvs-technology-<STUDENT_NAME>-sns-topic), scroll to the bottom, and click Create topic.
- Create a Subscription: You will be taken to your new topic's page. Scroll down to the Subscriptions tab and click Create subscription.
- Configure Email: Under the Protocol dropdown, select Email. In the Endpoint box, type your personal email address, then click Create subscription.
- Confirm the Email (Crucial): Go to your actual email inbox. You will receive a message from "AWS Notifications." You must open it and click the Confirm subscription link inside, otherwise, AWS will block your Lambda function from sending alerts to it.

Once confirmed, you can copy the **Topic ARN** (it looks like `arn:aws:sns:region:account-id: dvs-technology-<STUDENT_NAME>-sns-topic`) from the AWS console and paste it into your Lambda 2 Python code.

Architecture Overview

This pipeline guarantees that no data is lost during traffic spikes and protects your database from malformed payloads.

- **Ingestion:** A .json file lands in the dvs-<STUDENT_NAME>-retail-data-input S3 bucket.
- **Validation:** Lambda 1(data-validation-lambda) inspects the payload. Invalid files go to a dvs-<STUDENT_NAME>-retail-data-rejected bucket.
- **Buffering:** Valid records are split and sent individually to an SQS Queue.
- **Archiving:** A copy of the valid raw file is saved to a dvs-<STUDENT_NAME>-retail-data-validated bucket.
- **Database Write:** Lambda 2(dvs-retail-data-ddb-insertion-<studentname>) pulls records from SQS and inserts them into DynamoDB.
- **Notification:** Lambda 2 triggers an SNS topic to dispatch success emails.

2. Lambda 1: dvs-retail-data-validation-<studentname> : S3 to SQS (Ingestion)

This function triggers when a file is uploaded, saves a backup to the output bucket, and sends individual messages to SQS.

- **Permissions:** Attach `AmazonS3FullAccess` and `AmazonSQSFullAccess` to this function's execution role.
- Add an S3 Event Notification to trigger Lambda 1 on All object create events with a `.json` suffix.

```
"""
Pipeline: Data Ingestion & Reference Queueing (Claim Check Pattern)
Company: DVS Technology
Author: subrat
"""

import boto3
import json

s3 = boto3.client('s3')
sqs = boto3.client('sqs')

PROCESSED_BUCKET = 'dvs-<STUDENT_NAME>-retail-data-validated'
REJECTED_BUCKET = 'dvs-<STUDENT_NAME>-retail-data-rejected'
SQS_QUEUE_URL = 'YOUR_SQS_URL'

def lambda_handler(event, context):
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']

    # 1. Read JSON from the input S3 bucket
    file_content = s3.get_object(Bucket=bucket, Key=key)['Body'].read()
    data = json.loads(file_content)

    # 2. VALIDATION: Root key check
    if 'retail-purchases' not in data:
        print("Validation Failed: Missing 'retail-purchases' key.")
        s3.put_object(Bucket=REJECTED_BUCKET, Key=f"invalid-root-{key}", Body=file_content)
        return {"statusCode": 400, "body": "Rejected: Missing root key."}

    # 3. VALIDATION: Primary and Sort Key check
    for item in data['retail-purchases']:
        item_data = item.get('PutRequest', {}).get('Item', {})

        if 'customerid' not in item_data or 'purchasedate' not in item_data:
            print("Validation Failed: Missing required DynamoDB keys.")
            s3.put_object(Bucket=REJECTED_BUCKET, Key=f"missing-keys-{key}", Body=file_content)
            return {"statusCode": 400, "body": "Rejected: Missing required DB keys."}

    # 4. Save the verified copy to the processed bucket
    processed_key = f"processed-{key}"
    s3.put_object(Bucket=PROCESSED_BUCKET, Key=processed_key, Body=file_content)

    # 5. Send ONLY the location metadata to SQS
    sqs_payload = {
        "bucket": PROCESSED_BUCKET,
        "key": processed_key
    }
```

```

}

sqs.send_message(
    QueueUrl=SQS_QUEUE_URL,
    MessageBody=json.dumps(sqs_payload)
)

return {"statusCode": 200, "body": "File archived and location reference queued."}

```

3. Lambda 2: dvs-retail-data-ddb-insertion-<studentname> : SQS to DynamoDB & SNS

This function is triggered automatically by SQS to process the queue.

- **Permissions:** Grant AmazonSQSFullAccess, AmazonDynamoDBFullAccess, and AmazonSNSFullAccess, AmazonS3FullAccess
- Add an SQS Event Source Mapping in Lambda 2 to automatically poll the queue.

```

"""
Pipeline: Database Insertion & Alerting from S3 Reference
Company: DVS Technology
Author: subrat
"""
import boto3
import json

s3 = boto3.client('s3')
dynamodb = boto3.client('dynamodb', region_name='ap-south-1')
sns = boto3.client('sns', region_name='ap-south-1')

SNS_TOPIC_ARN = 'arn:aws:sns:ap-south-1:974268348508:dvs-technology-tashwik-sns-topic'
TABLE_NAME = 'retail-purchases'

def lambda_handler(event, context):
    for record in event['Records']:
        try:
            # 1. Parse S3 pointer from SQS message body
            s3_pointer = json.loads(record['body'])
            target_bucket = s3_pointer['bucket']
            target_key = s3_pointer['key']

            # 2. Download the validated file directly from processed S3 bucket
            response = s3.get_object(Bucket=target_bucket, Key=target_key)
            file_content = response['Body'].read().decode('utf-8')
            data = json.loads(file_content)

            # 3. Write items to DynamoDB
            for item in data[TABLE_NAME]:
                item_data = item['PutRequest']['Item']
                dynamodb.put_item(TableName=TABLE_NAME, Item=item_data)

            # 4. Dispatch Email Alert
            sns.publish(

```

```

        TopicArn=SNS_TOPIC_ARN,
        Subject="DVS Technology: New Purchase Processed",
        Message=f"Successfully inserted record for customer:
{item_data['customerid']}['S']}"
    )

    except dynamodb.exceptions.ProvisionedThroughputExceededException as e:
        print(f"DynamoDB Throttling Error on message {record['messageId']}: {str(e)}")
        raise e # Returns message to SQS for retry / DLQ fallback

    except Exception as e:
        print(f"Unexpected error processing message {record['messageId']}: {str(e)}")
        raise e

    return {"statusCode": 200, "body": "Processed S3 data and updated database successfully."}

```

HOMEWORK:

Implement Dead letter queue for rejected file inputs

Step 1: Create the Dead Letter Queue (DLQ)

First, you need a destination for the failed messages to go.

1. Open the **Amazon SQS Console**.
2. Click **Create queue**.
3. Name it something like `dvs-technology-<studentname>-dlq` (keep it the same type—Standard—as your main queue).
4. Leave all other settings as default and click **Create queue**.

Step 2: Configure the "Redrive Policy" on your Main Queue

Now, you tell your main queue to use this new DLQ after 3 failures.

1. In the SQS console, click on your **Main Queue** (the one Lambda 1 sends messages to).
2. Click **Edit** in the top right corner.
3. Scroll down to the **Dead-letter queue** section.
4. Select **Enabled**.
5. **Choose queue:** Select the `dvs-technology-dlq` you just created.
6. **Maximum receives:** Type 3.
7. Click **Save**.

How the Flow Works Now

Let's say a message arrives that is missing the `customerid` entirely.

1. **Attempt 1:** SQS hands the message to Lambda 2. DynamoDB rejects it. The `except` block catches the error, prints it to your logs, and triggers `raise e`. SQS takes the message back. `ReceiveCount` becomes 1.
2. **Attempt 2:** SQS hands it to Lambda again. It fails again. `raise e` is triggered. `ReceiveCount` becomes 2.
3. **Attempt 3:** SQS hands it to Lambda one last time. It fails again. `raise e` is triggered. `ReceiveCount` becomes 3.
4. **The Quarantine:** SQS sees that the `ReceiveCount` has hit the maximum limit of 3. Instead of giving it back to Lambda, SQS automatically moves the message into `dvs-technology-dlq` and deletes it from the main queue.